

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Khôi Nguyên
Nguyễn Tuấn Phùng
Hồ Bùi Văn Quang

ĐỒ ÁN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

Tp. Hồ Chí Minh, tháng 08/2021

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

Nguyễn Khôi Nguyên - 1753078

Nguyễn Tuấn Phùng - 1753087

Hồ Bùi Văn Quang - 1753093

**XÂY DỰNG HỆ THỐNG HỎI ĐÁP PHÁP
LUẬT
ĐỒ ÁN TỐT NGHIỆP**

**ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO**

GIÁO VIÊN HƯỚNG DẪN

TS. Nguyễn Trường Sơn

ThS. Nguyễn Thị Như Anh

Tp. Hồ Chí Minh, tháng 08/2021

Lời cảm ơn

Trong suốt thời gian học tập và rèn luyện tại Trường Đại học Khoa Học Tự Nhiên cho đến nay, chúng em đã nhận được rất nhiều sự quan tâm, giúp đỡ của quý Thầy Cô và bạn bè. Với lòng biết ơn sâu sắc và chân thành nhất, chúng em xin gửi đến Thầy Cô ở Khoa Công nghệ thông tin đã cùng với tri thức và tâm huyết của mình để truyền đạt vốn kiến thức quý báu cho chúng em trong suốt thời gian học tập tại trường.

Và đặc biệt, trong học kỳ này, Khoa đã tạo cơ hội cho chúng em được tiếp cận với Đồ án tốt nghiệp mà theo chúng em là rất hữu ích đối với chúng em cũng như tất cả các sinh viên thuộc các chuyên ngành Hệ Thống Thông Tin. Đó là đề tài "Áp dụng máy học vào trong hệ thống hỏi đáp pháp luật". Chúng em xin chân thành cảm ơn thầy Nguyễn Trường Sơn và cô Nguyễn Thị Như Anh đã tận tâm hướng dẫn chúng em qua từng buổi thảo luận về nghiệp vụ cũng như kiến thức bổ sung về chuyên ngành.

Chúng em xin bày tỏ lòng biết ơn đến ban lãnh đạo của Trường Đại học Khoa Học Tự Nhiên và các Khoa Phòng ban chức năng đã trực tiếp và gián tiếp giúp đỡ em trong suốt quá trình học tập và thực hiện đề tài này.

Với điều kiện thời gian cũng như kinh nghiệm còn hạn chế của một sinh viên, bài báo cáo này không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của các quý thầy cô để chúng em có điều kiện bổ sung, nâng cao kiến thức của mình, phục vụ tốt hơn công việc thực tế sau này.

Cuối cùng chúng em kính chúc quý thầy, cô dồi dào sức khỏe và thành công trong sự nghiệp cao quý. Chúng em xin chân thành cảm ơn!

Nhóm thực hiện

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Đồ án đáp ứng yêu cầu của Đồ án tốt nghiệp cử nhân CNTT.

Tp.HCM, ngày tháng năm 2021

Giáo viên hướng dẫn

NHẬN XÉT CỦA GIÁO VIÊN PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Đồ án đáp ứng yêu cầu của Đồ án tốt nghiệp cử nhân CNTT.

Tp.HCM, ngày tháng năm 2021

Giáo viên phản biện

Mục lục

Lời cảm ơn	1
Nhận xét của giáo viên hướng dẫn	1
Nhận xét của giáo viên phản biện	1
Đề cương chi tiết	1
Mục lục	1
1 MỞ ĐẦU	1
1.1 Giới thiệu chung	1
1.2 Mục tiêu đề tài	2
1.3 Nội dung của đề án	3
2 KHẢO SÁT HIỆN TRẠNG	4
2.1 Công Nghệ Sử Dụng	4
2.1.1 Thư viện BeautifulSoup	4
2.1.2 Phân tách từ bằng Python Vietnamese Toolkit . . .	5
2.1.3 Mô hình Transformer	7
2.1.4 Mô hình BERT	9
2.1.5 Sentence-transformer	10
2.1.6 Mô hình PhoBERT	11
2.1.7 Pre-trained model SimCSE	12
2.1.8 Elastic Search	12
2.1.9 Web Framework: Flask Framework	14
2.2 Bài Toán Liên Quan	16
2.3 Mô Tả Hệ Thống	17
3 Thu Thập Dữ Liệu - Scraping Data	19
3.1 Quy trình thực hiện Scraping Data	19

3.2	Chi tiết về dữ liệu	20
3.3	Kết quả	23
4	Kiến trúc phần Back-end	24
4.1	Xử lý dữ liệu thu thập	24
4.1.1	Phân loại từ trong câu hỏi	25
4.1.2	Mã hoá dữ liệu từ dataset	26
4.1.3	Lưu dữ liệu đã mã hoá vào Elastic Search	30
4.1.4	Kết quả	32
4.2	Xử lý câu hỏi từ người dùng	33
4.2.1	Tính toán mức độ tương đồng ngữ nghĩa giữa các câu	34
4.2.2	Trả về kết quả sau khi so sánh	36
5	Kiến trúc phần Front-end	37
5.1	Tạo Web server bằng Flask	37
5.2	Parse dữ liệu từ dữ liệu trả về từ Back-end	38
6	Kết luận	40
	Tài liệu tham khảo	41

Danh sách hình

2.1	Minh hoạ tách từ bằng PyVi	6
2.2	Mô hình tổng quát của cấu trúc của Transformer	8
2.3	Kích thước của 2 dạng BERT	9
2.4	Encode BERT	10
2.5	Flask logo	14
2.6	Mô tả hệ thống	17
3.1	Sơ đồ hoạt động của Scraping data từ website	19
3.2	Giao diện trang web Hỏi đáp pháp luật	21
3.3	Một block parent-item	21
3.4	Chi tiết bên trong mã nguồn parent-item	22
4.1	Quy trình lưu dữ liệu xuống Elastic Search	24
4.2	Tokenize một câu hỏi từ dataset	25
4.3	Quy trình thực hiện Encoding question	26
4.4	Encoders Input	27
4.5	Ma trận mã hoá dữ liệu của BERT[12]	29
4.6	Quy trình xử lý câu hỏi từ người dùng	33

Danh sách bảng

3.1	Bảng kiểu dữ liệu thu thập	20
4.1	Bảng kiểu dữ liệu thêm vào cơ sở dữ liệu	30

Chương 1

MỞ ĐẦU

Nội dung Chương 1 giới thiệu thực trạng chung của nhu cầu tra cứu cũng như đặt ra những câu hỏi liên quan đến vấn đề pháp luật ở Việt Nam, từ đó hình thành nên lý do và mục tiêu để thực hiện đề án.

1.1 Giới thiệu chung

Hiện nay, nhu cầu tìm kiếm cũng như hỏi đáp các vấn đề liên quan đến các lĩnh vực trong đời sống đang rất lớn, đặc biệt là sử dụng các dịch vụ tra cứu trực tuyến. Các lĩnh vực có nhu cầu tìm kiếm thông tin như công nghệ, chính trị, xã hội, luật pháp,... Đặc biệt là trong lĩnh vực luật pháp, điều mà trước đây chỉ có thể sử dụng phương pháp truyền thống là đến các cơ sở có khả năng pháp lý như văn phòng luật sư, các cơ sở có quyền về pháp lý,... mà người có nhu cầu muốn giải đáp thắc mắc có thể đến và đặt ra những câu hỏi cụ thể để từ đó, những chuyên gia có thể giải đáp được những câu hỏi cho họ.

Nhưng với thời điểm công nghệ 4.0 hiện nay, việc sử dụng internet đối với mỗi người chúng ta là điều thiết yếu trong cuộc sống. Vậy nên, việc xuất hiện các trang web, những dịch vụ trực tuyến nhằm giải đáp những vấn đề trên được hình thành. Tuy nhiên, có một số điểm yếu mà còn tồn tại trong hệ thống này là:

- Các câu hỏi được giải đáp bởi các chuyên gia và chỉ được lưu trữ dạng văn bản thuần.
- Khi cần tra cứu câu hỏi, người dùng cần phải nhập đúng chính xác câu hỏi hoặc chính xác từ khoá mà có câu hỏi mẫu đã được lưu trữ trong cơ sở dữ liệu của hệ thống.

- Nếu như người dùng cần tìm những câu hỏi có nội dung tương tự như câu hỏi đang cần hỏi thì cần phải biết được lĩnh vực mà câu hỏi đã được hỏi.

1.2 Mục tiêu đề tài

Với nhu cầu trao đổi thông tin hiện nay ngày càng cao cùng với sự phát triển của mạng Internet và công nghệ, hằng ngày con người luôn phải xử lý một lượng thông tin lớn. Do đó việc hỏi đáp mang tính thiết yếu hơn trong cuộc sống, và nhiều trang web, hệ thống hỏi đáp đã được xây dựng nên để đáp ứng nhu cầu đó. Nhưng trong số đó, vẫn còn nhiều hệ thống hỏi đáp chưa thể đưa ra câu trả lời ngay lập tức được hoặc phải do chính nhân tố con người trả lời một cách thủ công. Đặc biệt khi nói về lĩnh vực Pháp luật – lĩnh vực mang tính chất khô khan và cần nhiều sự chính xác và nhanh chóng khi đưa ra câu trả lời đối với một hệ thống hỏi đáp thì hiện nay vẫn chưa có nhiều hệ thống đáp ứng được.

Đề tài hướng đến mục tiêu là xây dựng một hệ thống hỏi đáp tự động tiếng Việt cho một lĩnh vực cụ thể là về pháp luật Việt Nam. Bằng cách áp dụng công nghệ nhiều hơn, ở đây chính là công nghệ trí tuệ nhân tạo giúp nâng cao hiệu quả, tốc độ và sự chính xác trong câu trả lời mà hệ thống đưa ra. Với hướng tiếp cận xử lý ngôn ngữ tự nhiên, sử dụng các Pre-trained Model (mô hình tiền huấn luyện) để phân tích ngữ nghĩa của các câu hỏi và cho ra kết quả chính xác nhất thì việc xây dựng một hệ thống về hỏi đáp Pháp luật Việt Nam là một đề tài hoàn toàn khả thi.

1.3 Nội dung của đề án

Đề án có tổng cộng 6 chương với nội dung chính như sau:

Chương 1: Mở đầu

Nội dung chương 1 giới thiệu về thực trạng về nhu cầu trong thực tế của việc hỏi đáp các vấn đề liên quan đến luật pháp ở Việt Nam, đưa ra các điểm yếu trong phương pháp truyền thống và đưa ra mục tiêu, phương pháp áp dụng của đề án vào trong ứng dụng.

Chương 2: Khảo sát hiện trạng

Nội dung của chương 2 là đưa ra các cơ sở lý thuyết để áp dụng vào trong đề tài, chỉ ra các công nghệ được áp dụng và phối hợp với nhau để giải quyết được vấn đề cần phải giải quyết trong đề tài nghiên cứu.

Chương 3: Thu Thập Dữ Liệu

Nội dung chương 3 là trình bày về tổng quan của hệ thống và quy trình để xây dựng nên hệ thống về phần Back-end và Front-end.

Chương 4: Kiến trúc phần Back-end

- Trình bày chi tiết về quá trình thu thập dữ liệu pháp luật
- Phương pháp xây dựng cơ sở dữ liệu và lưu trữ dữ liệu
- Mô hình đào tạo dữ liệu phương pháp phân tích đưa ra được thông tin chuẩn xác từ mô hình.

Chương 5: Kiến trúc phần Front-end

Trình bày về trình tự và cách để đưa dữ liệu được trả về ở phần Back-end hiển thị lên giao diện người dùng.

Chương 6: Kết luận và hướng phát triển Nội dung chương 5 là tóm tắt lại kết quả đạt được sau khi hoàn thành đề tài, ý nghĩa của đề tài và định hướng phát triển của đề tài trong tương lai

Chương 2

KHẢO SÁT HIỆN TRẠNG

2.1 Công Nghệ Sử Dụng

2.1.1 Thư viện BeautifulSoup

BeautifulSoup[7] là một thư viện của Python nhằm hỗ trợ để bóc tách dữ liệu ra khỏi các file HTML. Nó hoạt động cùng với các **parser** để điều hướng, tìm kiếm, chỉnh sửa trong parse tree. Các parser khác nhau như là: `html.parser`, `lxml`, `html5lib`.⁷

Parser **lxml** là một loại parser hỗ trợ trong việc phân tích HTML để phục vụ cho công việc scraping dữ liệu.

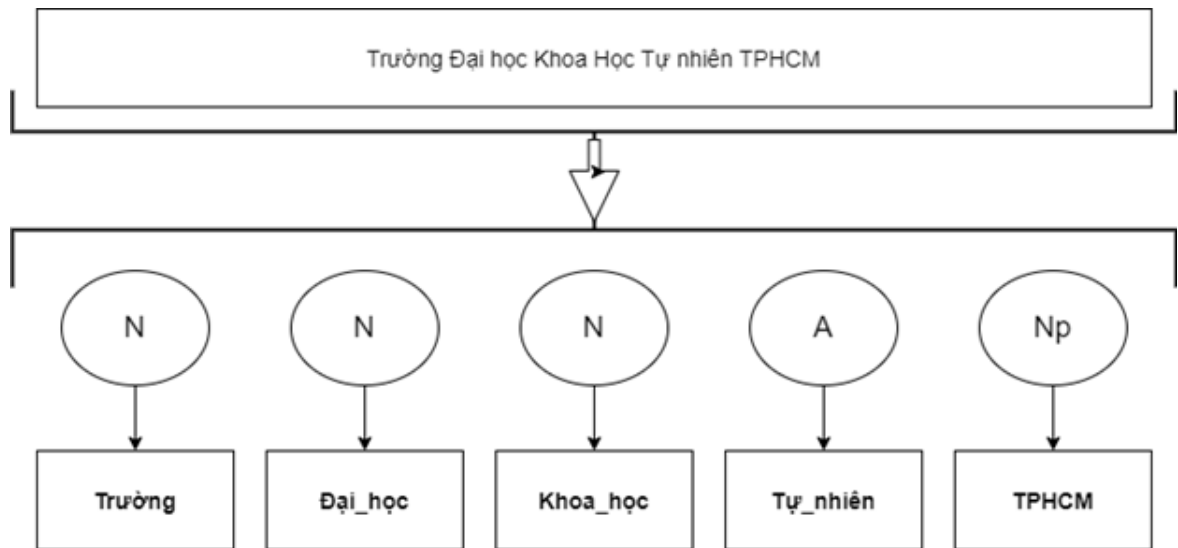
Các Object cần quan tâm khi sử dụng BeautifulSoup là:

- Tag: tương ứng với một thẻ tag trong tài liệu HTML.
- NavigableString: là một chuỗi tương ứng với một bit văn bản có chứa trong một tag.
- BeautifulSoup: là một đối tượng đại diện cho toàn bộ tài liệu.
- Comment: là một dạng đặc biệt của NavigableString, cho phép truy cập vào các comment của một file HTML.

2.1.2 Phân tách từ bằng Python Vietnamese Toolkit

Python Vietnamese Toolkit(PyVi)[14] là một công cụ xử lý ngôn ngữ tự nhiên dùng cho tiếng Việt với các hàm xử lý như:

- Tokenization: tách từ, cụm từ trong văn bản
- POS tagging: gán nhãn từ loại, các loại từ bao gồm:
 - A – Adjective: tính từ
 - C – Coordinating conjunction: liên từ
 - E – Preposition: giới từ
 - I – Interjection: thán từ
 - L – Determiner: từ hạn định
 - M – Numeral: số
 - N - Common noun: danh từ chung
 - Nc - Noun Classifier: phân loại danh từ
 - Ny - Noun abbreviation: các danh từ viết tắt
 - Np - Proper noun: danh từ riêng
 - Nu - Unit noun: danh từ đơn vị
 - P – Pronoun: đại từ
 - R – Adverb: trạng từ
 - S - Subordinating conjunction: liên từ phụ thuộc
 - T - Auxiliary, modal words: động từ khiếm khuyết
 - V – Verb: động từ
 - X – Unknown: các từ không xác định được
 - F - Filtered out (punctuation): các từ được lọc ra bởi dấu chấm câu
- Accents removal: bỏ dấu từ
- Accents adding: thêm dấu từ



Hình 2.1: Minh hoạ tách từ bằng PyVi

Sau đây là một đoạn code mẫu mô tả cách sử dụng và kết quả trả về của PyVi:

```
from pyvi import ViTokenizer, ViPosTagger

from pyvi import ViUtils

ViTokenizer.tokenize(u"Trường Đại học Khoa học Tự nhiên TP HCM")

ViPosTagger.postagging(ViTokenizer.tokenize(u" Trường Đại học Khoa học Tự
nhiên TP HCM "))

ViUtils.remove_accents(u" Trường Đại học Khoa học Tự nhiên TP HCM")

ViUtils.add_accents(u'truong dai hoc khoa hoc tu nhien TP HCM')
```

Khi chạy đoạn code trên, ta sẽ thu được các kết quả như:

Tokenize (tách từ):

```
Trường Đại_học Khoa_học Tự_nhiên TP HCM
```

POS_tagging (gán nhãn từ loại): khi thực hiện POS tagging thì hàm sẽ thực hiện tách từ (Tokenize) trước khi trả về kết quả. Kết quả trả về gồm mảng 2 chiều với thứ tự gán nhãn tương ứng với nhau:

```
(['Trường', 'Đại_học', 'Khoa_học', 'Tự_nhiên', 'TPHCM'], [ 'N', 'N', 'N', 'A',  
'Np'])
```

Accents removal (bỏ dấu):

```
Truong Dai hoc Khoa hoc Tu nhien TPHCM
```

Accents adding(Thêm dấu):

```
Trường Đại học Khoa học Tự nhiên TPHCM
```

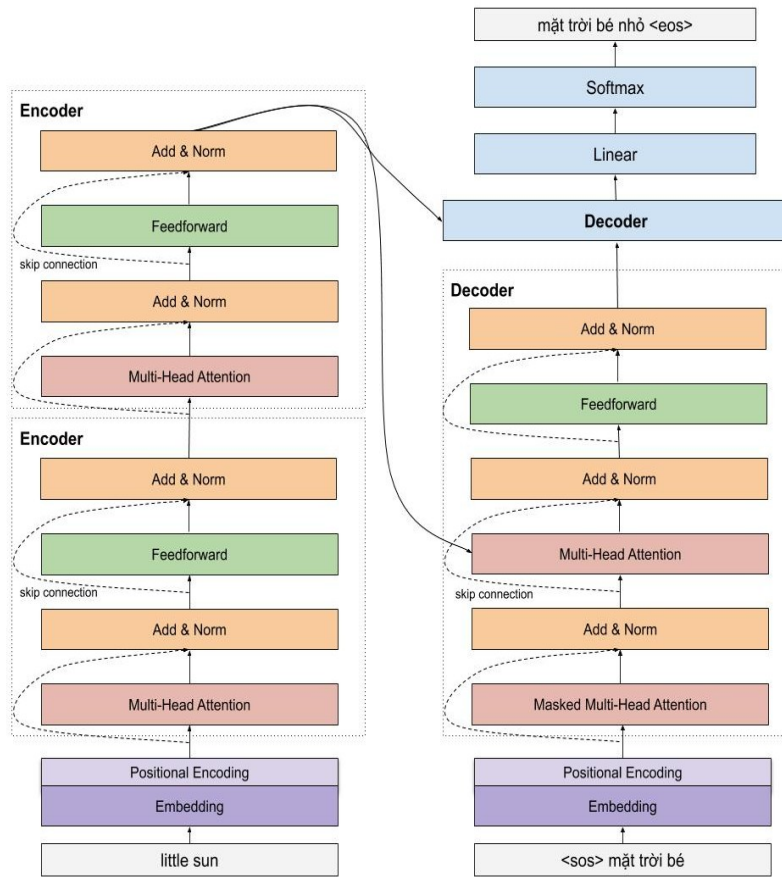
2.1.3 Mô hình Transformer

Trước khi Transformers xuất hiện, đa số các tác vụ xử lý ngôn ngữ tự nhiên sử dụng kiến trúc của Recurrent Neural Networks(RNNs) – Mạng nơ-ron hồi quy. Điểm yếu của RNNs là khó nắm bắt được sự phụ thuộc xa giữa các từ trong câu và tốc độ huấn luyện mô hình chậm do phải xử lý tuần tự. Transformers có thể giải quyết các vấn đề này.

Transformers ra đời nhằm giải quyết vấn đề chuyển đổi chuỗi dữ liệu hoặc dịch máy, chẳng hạn như các bài toán như nhận dạng giọng nói, chuyển văn bản thành giọng nói,...

Kiến trúc của Transformers bao gồm hai phần là Encoder và Decoder, Encoder dùng để học, chuyển hoá vector biểu diễn của câu văn bản và Decoder sẽ thực hiện biểu diễn vector đó dưới ngôn ngữ của người dùng.

Ưu điểm của Transformers là mô hình này có khả năng xử lý song song cho các từ trong câu văn bản, gia tăng hiệu suất và giảm thời gian huấn luyện.



Hình 2.2: Mô hình tổng quát của cấu trúc của Transformer

Ý tưởng của mô hình Transformers là áp dụng cơ chế Attention. Chính thành phần Multi-Head Attention trong các lớp Encoder cho phép mô hình mã hoá một từ có thể sử dụng thông tin của những từ liên quan đến nó.

2.1.4 Mô hình BERT

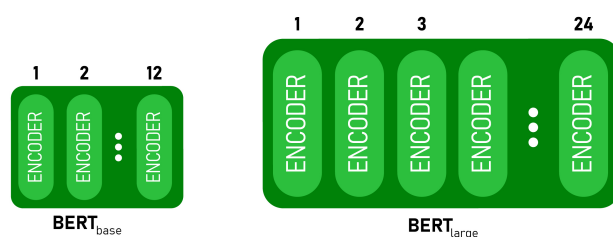
BERT là viết tắt của “Bidirectional Encoder Representations from Transformers”, là một mô hình học sâu, học biểu diễn vector đại diện cho câu theo ngữ cảnh 2 chiều.

Một trong những khó khăn khi xử lý ngôn ngữ tự nhiên đó là việc thiếu hụt dữ liệu gán nhãn. Hầu hết các dữ liệu đều được gán nhãn thủ công bởi con người, vì vậy, số lượng mẫu dữ liệu chất lượng không nhiều. Do đó, để giải quyết vấn đề này, các mô hình xử lý ngôn ngữ tự nhiên sử dụng cơ chế tiền xử lý dữ liệu, cơ chế này tạo ra một mô hình chung được huấn luyện từ số lượng lớn dữ liệu không gán nhãn, chẳng hạn như các mô hình Word2Vec, Glove,... Tuy nhiên, các mô hình này thường không biểu diễn chính xác ngữ nghĩa của từ trong ngữ cảnh câu văn bản, dẫn đến khó khăn trong việc xác định nghĩa của một từ đang được đề cập trong ngữ cảnh nào.

BERT có khả năng tạo ra mô hình ngôn ngữ cho phép các hệ thống tìm kiếm hiểu rõ hơn về ngữ nghĩa của từ trong câu dựa vào xem xét mối liên hệ giữa các từ trước và sau của từ đó.

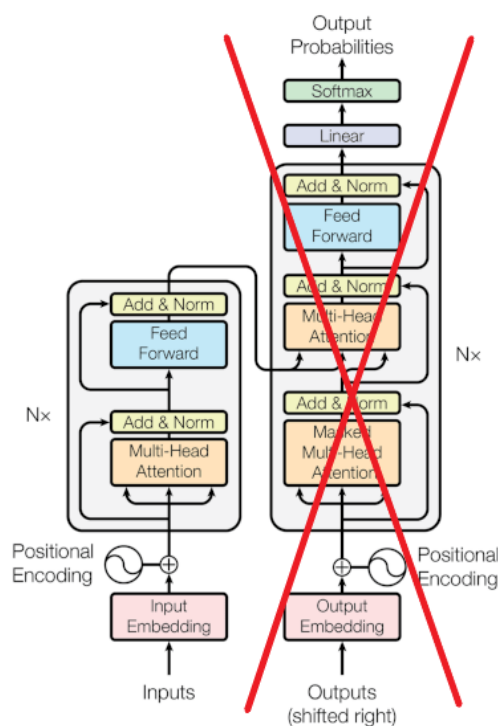
BERT được xây dựng dựa trên Transformers với hai biến thể:

- BERT_{BASE} gồm 12 lớp Encoder, kích thước của các lớp ẩn là 768 với 110 triệu tham số.
- BERT_{LARGE} gồm 24 lớp Encoder, kích thước của các lớp ẩn là 1024 với 340 triệu tham số



Hình 2.3: Kích thước của 2 dạng BERT

Trong khi đó kiến trúc của Transformers chỉ có 6 lớp Encoder, cho thấy khả năng xử lý của BERT mang lại hiệu suất cao hơn.



Hình 2.4: Encode BERT

Kiến trúc của BERT khác với Transformers ở chỗ nó chỉ dùng thành phần Encoder của Transformers vì mục tiêu sử dụng BERT là để tạo ra được một mô hình thực hiện các tác vụ NLP khác nhau. Việc sử dụng Encoder cho phép BERT mã hóa thông tin ngữ nghĩa, cú pháp của câu trong quá trình nhúng.

BERT không sử dụng thành phần Decoder của Transformers nên kết quả đầu ra của nó là một embedding – một không gian vector dùng để biểu diễn dữ liệu có thông tin về mối liên hệ, sự tương đồng về mặt ngữ nghĩa, ngữ cảnh của dữ liệu, trong đó bao gồm nhiều chiều và các từ có sự tương đồng sẽ có vị trí gần nhau. Vì kết quả đầu ra của BERT là word embedding nên người sử dụng có thể làm các tác vụ khác nhau. Người dùng có thể sử dụng các kỹ thuật để tính sự tương đồng, chẳng hạn như Cosine Similarity để so sánh giữa các vector đại diện và trả về điểm số tương đồng.

2.1.5 Sentence-transformer

Sentence Transformers[9] là 1 framework dùng để Embedd (nhúng) những câu văn, đoạn văn, hình ảnh để tạo ra các Embeddings có ý nghĩa

về mặt ngữ nghĩa, được sử dụng để áp dụng cho các bài toán, ứng dụng Semantic Search (tìm kiếm ngữ nghĩa) hoặc là Multi-lingual Zero-shot Classification (phân loại Zero-shot đa ngôn ngữ). Sentence Transformers phương pháp để tính toán, tạo ra các vector biểu diễn cho các câu văn, đoạn văn bản hay là một hình ảnh. Các mô hình như BERT, RoBERTa, XLM-RoBERTa,... đều dựa trên framework Sentence Transformers này. Các đoạn văn bản sau khi được Vector hóa có thể tìm được sự tương đồng về mặt ngữ nghĩa với nhau bằng cách dùng phương pháp tính Cosine Similarity.

Như vậy với Sentence Transformers, ta có thể:

- Dùng Cosine Distance để tính độ tương đồng về ngữ nghĩa của các câu, áp dụng để phân nhóm câu theo ngữ nghĩa (Semantic Clustering), dùng trong Hệ thống truy vấn thông tin theo ngữ nghĩa (Semantic Information Retrieval), ...
- Sử dụng như Pre-trained Model trong các Transfer Learning

Ví dụ về sử dụng mô hình Sentence Transformer để nhúng (Embedd) các câu văn:

2.1.6 Mô hình PhoBERT

PhoBERT[8],[10] là một pre-trained model dựa trên BERT, được xây dựng để huấn luyện riêng cho ngôn ngữ tiếng Việt, dựa trên kiến trúc RoBERTa – model tương tự BERT nhưng có phần tối ưu quá trình tiền huấn luyện cho hiệu suất cao do kiến trúc này đã bỏ đi tác vụ Next Sentence Prediction.

PhoBERT cũng có hai kích thước là PhoBERT_{BASE} với 12 transformers block và PhoBERT_{LARGE} với 24 transformers block. PhoBERT được huấn luyện từ 20GB dữ liệu về Tiếng Việt bao gồm 1GB Vietnamese Wikipedia corpus và 19GB Vietnamese news corpus.

PhoBERT sử dụng RDRSegmenter của VnCoreNLP để tách từ cho dữ liệu đầu vào trước khi qua BPE encoder.

PhoBERT tương tự BERT, chỉ khác là dùng cho việc xử lý ngôn ngữ

tiếng Việt nên kiến trúc của PhoBERT cũng thừa hưởng thành phần Encoder của mô hình Transformers, không bao gồm Decoder.

2.1.7 Pre-trained model SimCSE

SimCSE[6] là một pre-trained model sử dụng PhoBERT, dùng để mã hóa câu, đoạn văn bản dữ liệu đầu vào. SimCSE cho hiệu suất huấn luyện cao hơn, làm việc đồng thời với cả hai loại dữ liệu đã được gắn nhãn hoặc không.

Việc áp dụng mô hình SimCSE là cần thiết vì đây là mô hình sử dụng các phương pháp kỹ thuật hiện đại nhất trong tác vụ xử lý dữ liệu đầu vào. Thêm vào đó, với tác vụ đánh giá sự tương đồng ngữ cảnh giữa các câu văn bản, SimCSE mang lại kết quả khá cao so với BERT.

2.1.8 Elastic Search

Elasticsearch là một công cụ tìm kiếm dựa trên nền tảng Apache Lucene. Nó cung cấp một bộ máy tìm kiếm dạng phân tán, có đầy đủ công cụ với một giao diện web HTTP có hỗ trợ dữ liệu JSON.

Elasticsearch chạy trên server riêng và đồng thời giao tiếp thông qua RESTful do vậy nên nó không phụ thuộc vào client viết bằng gì hay hệ thống hiện tại của bạn viết bằng gì. Nên việc tích hợp nó vào hệ thống bạn là dễ dàng, bạn chỉ cần gửi request http lên là nó trả về kết quả.

Các khái niệm cần phải biết khi sử dụng Elastic Search[5] là:

- Document: là một Object JSON, có thể xem là đơn vị lưu trữ nhỏ nhất của Elastic Search
- Index: được xem như là một "database" khi quy chiếu sang với cơ sở dữ liệu quan hệ.
- Shard: là khối các Elastic Search với mục đích mở rộng, nâng cấp.
- Node: Là trung tâm hoạt động của Elasticsearch. Là nơi lưu trữ dữ liệu, tham gia thực hiện đánh index của cluster cũng như thực hiện các thao tác tìm kiếm.

- Cluster: là tập hợp các nodes hoạt động cùng với nhau. Mỗi Cluster có một Node chính được chọn một cách tự động và có thể thay thế nếu có sự cố xảy ra.

Chức năng chính của Cluster đó chính là quyết định xem shards nào được phân bổ cho node nào và khi nào thì di chuyển các Cluster để cân bằng lại Cluster.

Ưu điểm của Elasticsearch:

- Tìm kiếm dữ liệu rất nhanh chóng, mạnh mẽ dựa trên Apache Lucene (near-realtime searching)
- Có khả năng phân tích dữ liệu (Analysis data)
- Khả năng mở rộng theo chiều ngang
- Hỗ trợ tìm kiếm mờ (fuzzy), tức là từ khóa tìm kiếm có thể bị sai lỗi chính tả hay không đúng cú pháp thì vẫn có khả năng elasticsearch trả về kết quả tốt.
- Hỗ trợ Structured Query DSL (Domain-Specific Language), cung cấp việc đặc tả những câu truy vấn phức tạp một cách cụ thể và rõ ràng bằng JSON.
- Hỗ trợ nhiều Elasticsearch client như Java, PHP, Javascript, Ruby, .NET, Python

Nhược điểm của Elastic Search:

- Elasticsearch được thiết kế cho mục đích search, do vậy với những nhiệm vụ khác ngoài search như CRUD thì elastic kém thế hơn so với những database khác như MongoDB, Mysql Do vậy người ta ít khi dùng elasticsearch làm database chính, mà thường kết hợp nó với 1 database khác.
- Trong elasticsearch không có khái niệm database transaction , tức là nó sẽ không đảm bảo được toàn vẹn dữ liệu trong các hoạt động Insert, Update, Delete. Tức khi chúng ta thực hiện thay đổi nhiều bản ghi

nếu xảy ra lỗi thì sẽ làm cho logic của mình bị sai hay dẫn tới mất mát dữ liệu. Đây cũng là 1 phần khiến elasticsearch không nên là database chính.

- Không thích hợp với những hệ thống thường xuyên cập nhật dữ liệu. Sẽ rất tốn kém cho việc đánh index dữ liệu.

2.1.9 Web Framework: Flask Framework

1. Flask Framework:



Hình 2.5: Flask logo

- Flask là một Web Frameworks, được xây dựng bằng ngôn ngữ lập trình Python.
- Flask dựa trên bộ công cụ **Werkzeug WSGI** và template engine **Jinja2**:
 - **WSGI** (Web Server Gateway Interface): WSGI là Giao diện Cổng Máy chủ Web, WSGI mô tả cách máy chủ web giao tiếp với các ứng dụng web và cách các ứng dụng web có thể được kết nối với nhau khi xử lý một yêu cầu.
 - **Werkzeug**: Werkzeug là một bộ công cụ WSGI thực hiện các yêu cầu, đối tượng phản hồi và các chức năng tiện ích, cho phép xây dựng một Web Framework trên đó. Flask

framework sử dụng Werkzeug làm một trong những cơ sở của nó.

- **jinja2**: jinja2 là một template engine phổ biến cho Python, được sử dụng để hiển thị dữ liệu cho các trang web. Jinja2 cho phép chuyển các biến Python vào các HTML template – thuận tiện cho việc đổ dữ liệu từ Back End đến Front End.
- Flask cung cấp các thư viện, công cụ, cơ chế để xây dựng các Website từ nhỏ đến lớn, từ đơn giản tới phức tạp như: các API nhỏ, trang blog, trang web thương mại điện tử,...
- Flask thuộc loại Micro-framework: có ưu điểm là rất nhẹ và nhỏ, không phụ thuộc vào thư viện bên ngoài nên rất ít lỗi, do đó dễ dàng đề phòng, phát hiện và xử lý các lỗi bảo mật

2. Ví dụ web API đơn giản xây dựng bằng Flask[13]:

- Sau đây là một đoạn code để tạo một Route của một website

```
1 from flask import Flask
2 app = Flask(__name__)
3 @app.route('/hello')
4 def hello_world():
5     return 'Hello, World!'
6 if __name__ == '__main__':
7     app.run(host='0.0.0.0', port=5001)
```

- Để khởi tạo instance của class, dòng code dưới sẽ tạo một web server điều hướng các request từ client đến ứng dụng của chúng ta, nó cũng là chính là một ứng dụng WSGI(Web Server Gateway Interface):

```
1 app = Flask(__name__):
```

- Route decorator cho Flask biết được url nào ứng với hàm nào, Web Server dựa vào đường dẫn và route decorator để chuyển yêu cầu tới hàm xử lý tương ứng *hello_world()*, hàm này đóng vai trò làm ứng dụng web (web application).

```
1 @app.route('/hello')
```


- Flask app được chạy với lệnh `app.run()`. Các tham số trong hàm run:

```
1 if __name__ == '__main__':  
2     app.run(host='0.0.0.0', port=5001)
```

- **Host= '0.0.0.0'**: Địa chỉ IP để ứng dụng Flask lắng nghe các kết nối, mặc định giá trị của tham số này là '127.0.0.1' và ứng dụng flask chỉ chấp nhận các kết nối từ máy local. chuyển giá trị này thành '0.0.0.0' để ứng dụng chấp nhận các kết nối từ bên ngoài.
- **port=5001**: Giá trị của port là một số nguyên, giá trị mặc định bằng 5001. Là cổng được mở trên máy mà ứng dụng Flask chạy, cần chọn cổng mà chưa được mở trên máy để tránh gặp lỗi khi chạy.

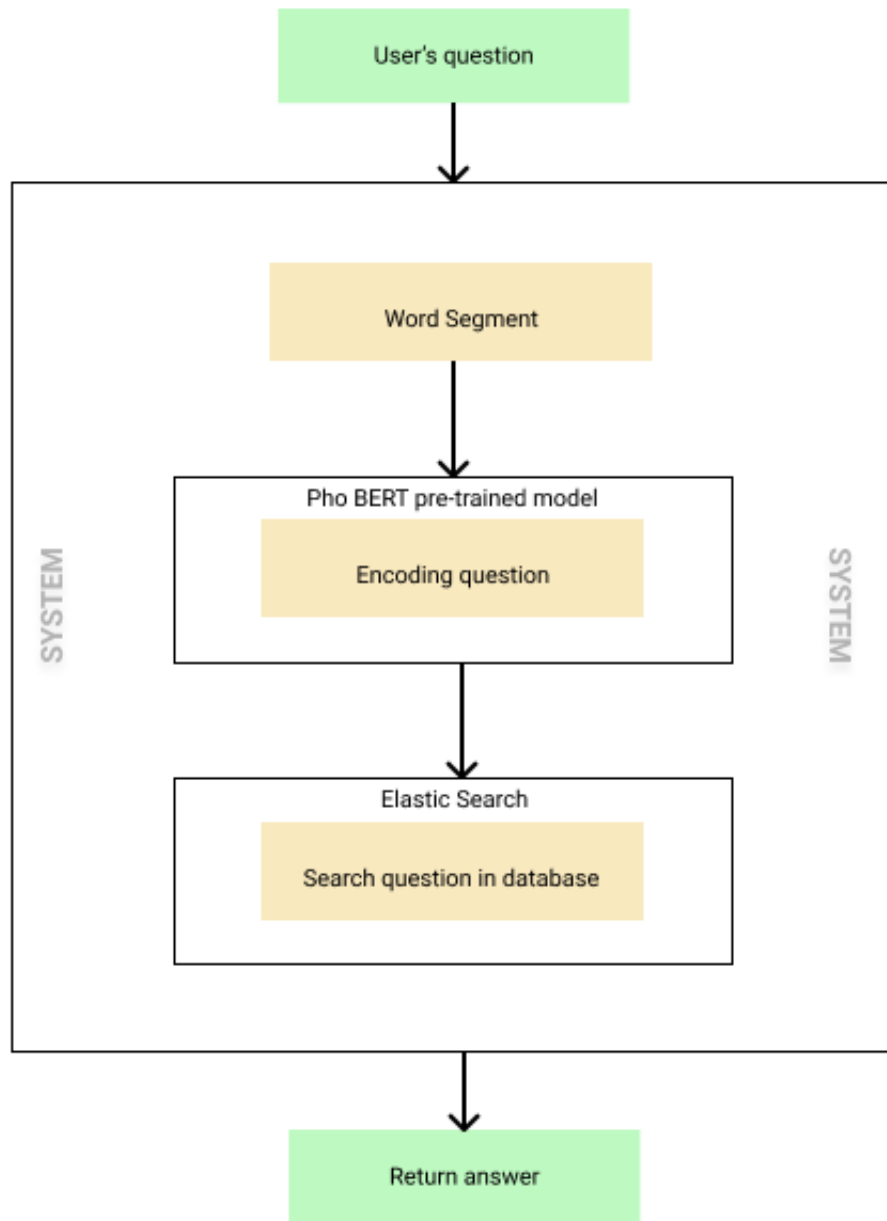
2.2 Bài Toán Liên Quan

Qua quá trình tìm kiếm, tìm hiểu thì nhóm rút ra được để giải quyết được yêu cầu của hệ thống thì sẽ cần giải được các bài toán nhỏ liên quan như sau:

- Thu thập dữ liệu về pháp luật từ các trang web chuyên về pháp luật
- Xử lý ngôn ngữ tự nhiên
- Mã hoá dữ liệu câu hỏi
- So sánh tìm ra sự tương đồng giữa các câu

2.3 Mô Tả Hệ Thống

Từ những mô tả và những công nghệ được nêu trên, ta có sơ đồ của hệ thống hỏi đáp pháp luật như sau:



Hình 2.6: Mô tả hệ thống

Mô hình của hệ thống có 5 quá trình như sau

- User's question: Người dùng sẽ cung cấp câu hỏi trên web site, từ đó, câu hỏi sẽ được gửi đến phía Back-end để thực hiện quá trình phân tích và tìm kiếm câu trả lời trong cơ sở dữ liệu.
- Word Segment: Đây là bước thực hiện đầu tiên khi câu hỏi được gửi đến phía Back-end, câu hỏi của người dùng sẽ được thực hiện phân tách từ theo ngữ nghĩa để có thể phân tích bước đầu xác định ý nghĩa câu hỏi người dùng.
- Encoding Question: Ở bước này, sau khi câu hỏi của người dùng đã được phân tách từ, câu hỏi sẽ được hệ thống mã hoá để có cùng với dạng dữ liệu với dữ liệu có trong cơ sở dữ liệu.
- Elastic Search: Tiến hành tìm kiếm câu hỏi của người dùng trong bộ câu hỏi được thu thập có trong cơ sở dữ liệu bằng Elastic Search.
- Respond answer: Từ kết quả trả về sau khi tìm kiếm trong hệ thống, câu trả lời cho câu hỏi của người dùng cùng với các bộ câu hỏi và câu trả lời có sự tương tự với câu hỏi của người dùng đã nhập vào trong hệ thống sẽ được gửi lên và hiển thị trên website cho người dùng.

Trong 3 chương tiếp theo, nhóm sẽ tiếp tục trình bày rõ từng bước mà hệ thống sẽ thực hiện, hoạt động, kết quả của từng bước có trong hệ thống.

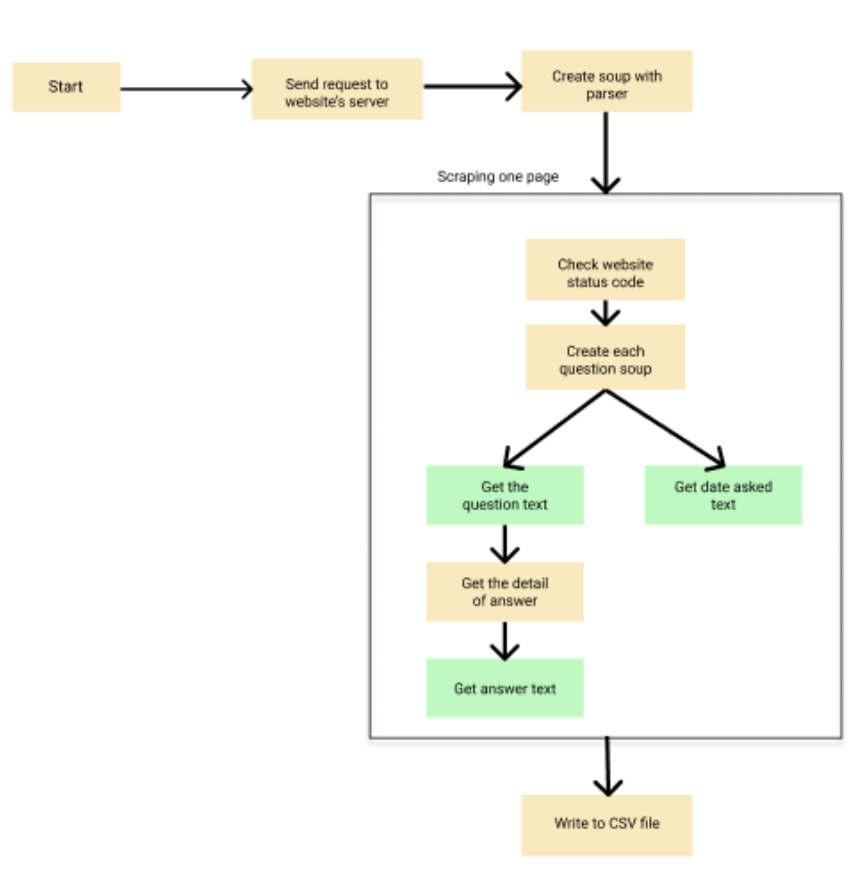
Chương 3

Thu Thập Dữ Liệu - Scraping Data

3.1 Quy trình thực hiện Scraping Data

Ở chương này, vấn đề cần phân tích là công việc đầu tiên cần phải làm trước tiên để bắt đầu cho các công việc sau này. Việc thu thập dữ liệu(**Scraping data**)[7] là bước đầu tiên trong mô hình của hệ thống.

Dưới đây là mô hình hoạt động của công việc Scraping data:



Hình 3.1: Sơ đồ hoạt động của Scraping data từ website

Qua các bước trong sơ đồ, ta thấy được rằng việc thu thập dữ liệu này là một vòng lặp hoạt động bóc tách các văn bản có trong một file HTML, cụ thể là:

- File HTML chứa danh sách câu hỏi, mô tả của câu hỏi, thời gian trả lời câu hỏi
- File HTML chứa thông tin câu hỏi, mô tả câu hỏi, câu trả lời cho câu hỏi

Qua tìm kiếm và lựa chọn thì nhóm quyết định sẽ tiến hành Scraping data ở website cung cấp dịch vụ hỏi đáp, cung cấp thông tin về pháp luật là **Thư viện Pháp Luật**[11] để tiến hành lấy dữ liệu về câu hỏi và trả lời làm bộ dữ liệu chính cho đề án.

3.2 Chi tiết về dữ liệu

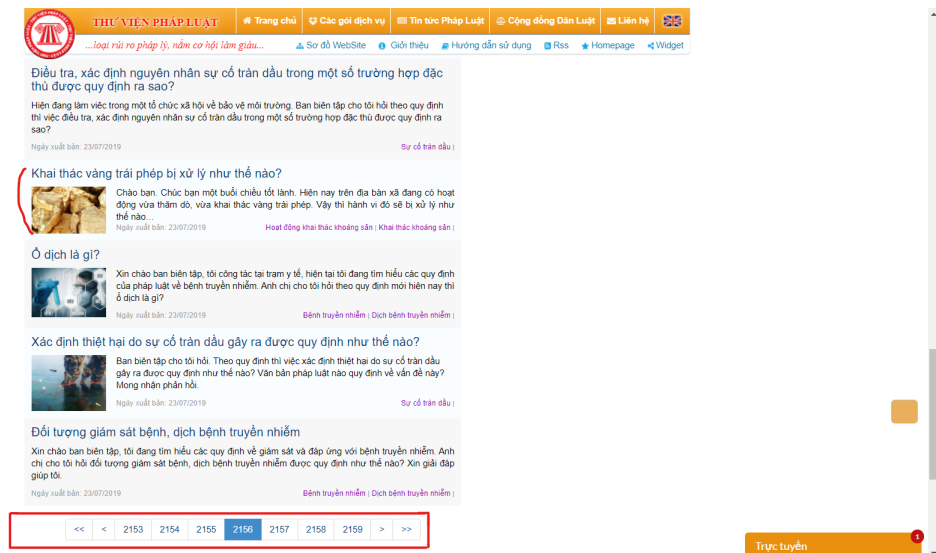
Sau khi thống nhất về dữ liệu tiến hành thu thập, nhóm đưa ra bảng kiểu dữ liệu sau:

Tên thuộc tính	Kiểu dữ liệu
Question	text
Answer	text
Date-asked	text

Bảng 3.1: Bảng kiểu dữ liệu thu thập

Tiếp đến ta cần phải phân tích file HTML chứa danh sách các câu hỏi, việc cần làm là tìm ra được các Title của các câu hỏi ở đâu và URL dẫn đến trang chứa thông tin câu trả lời. Ở đây, nhóm đã tìm ra được thông tin chứa câu hỏi cũng như câu trả lời nằm ở trong các tag HTML là:

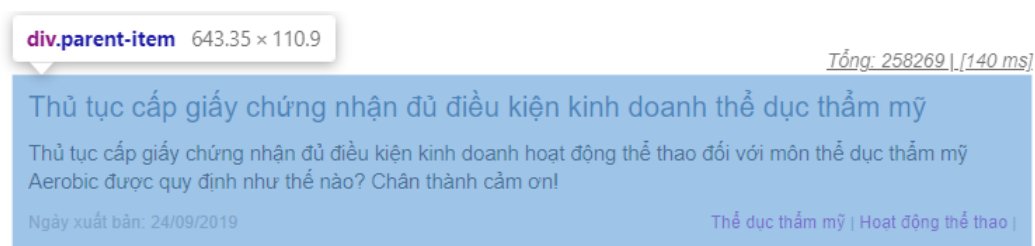
- Câu hỏi : class 'tieu-de'
- Ngày hỏi câu hỏi: span 'ngayhoi'
- Câu trả lời: div 'cautraloi'



Hình 3.2: Giao diện trang web Hỏi đáp pháp luật

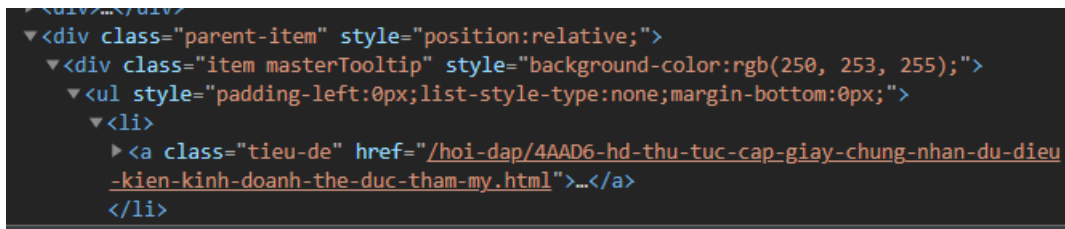
Qua giao diện của website thì các câu hỏi sẽ hiển thị ở dạng danh sách các câu hỏi, chia thành rất nhiều trang.

Để có thể lấy được data từ các câu hỏi khác nhau, ta cần phải lấy được thông tin của một thành phần nữa đó là URL của câu hỏi. Và URL này cũng được lấy bằng cách bóc tách dữ liệu từ file HTML của một trang. Sau khi có được Soup của 1 trang danh sách câu hỏi, ta cần tìm trong Soup đây thông tin của **parent-item** block, vì từ trong các block này sẽ chứa thông tin câu hỏi được hỏi và chứa cả URL dẫn tới trang chứa câu trả lời cho câu hỏi này.



Hình 3.3: Một block parent-item

Khi mở mã nguồn của trang và tìm tới đoạn của các parent-item thì ta sẽ thấy được thông tin của URL dẫn đến từng trang riêng biệt chứa thông tin của từng câu hỏi và câu trả lời như sau:



Hình 3.4: Chi tiết bên trong mã nguồn parent-item

Vậy nên, để lấy được thông tin của các URL của từng câu hỏi thì ta tiến hành tìm các *href* trong class *tieu-de*

```

1 parent_element=soup.find_all('div', class_='parent-item')
2 for element in parent_element:
3     link = 'https://hoidap.thuvienphapluat.vn' + str(element.a['href'])

```

Sau khi có được URL của từng câu hỏi rồi, chúng ta sẽ tiếp tục việc lấy dữ liệu của câu hỏi, câu trả lời, ngày hỏi của từng câu hỏi qua các thẻ tag *tieu-de*, *ngayhoi*, *cautraloi* trong từng URL đã lấy được ở trên. Sau đây là đoạn mã nguồn lấy thông tin của các thành phần trên.

```

1 if(get_page2.status_code == 200):
2     title = element.find('a', class_='tieu-de').h4.text
3     soup2 = BeautifulSoup(get_page2.text, 'lxml')
4     div_answer = soup2.find_all('div', class_='cautraloi')
5     div_date_asked = element.find('span', class_='ngayhoi')

```

Sau khi kiểm tra URL có sẵn sàng hay không bằng cách kiểm tra *status_code* của respond từ server website, nếu bằng 200 thì thực hiện các bước tiếp theo là tìm các thẻ tag câu hỏi, câu trả lời và ngày hỏi trong file HTML.

Sau đấy thì cần lưu lại từng cặp câu hỏi và câu trả lời vào trong file CSV. Lý do nhóm chọn sử dụng file CSV là để lưu xuống file những đoạn text data rất nhanh, định dạng cấu trúc file dễ dàng hơn so với file JSON.

Vì các bước công việc trên là để thực hiện lấy toàn bộ text data từ 1 trang của 1 câu hỏi, vậy nên để

Sau đấy từ file CSV convert sang file cấu trúc JSON để có thể thuận tiện hơn trong việc gửi lại dữ liệu lên website.

```

1 with open('/home/lan1', mode='a', newline='', encoding='utf-8') as f:
2     f = csv.DictWriter(f, fieldnames=csv_columns)
3     f.writerow({'Question': title,
4                 'Answer': p_ans.text,
5                 'Date-asked': div_date_asked.text})

```

Vì các công việc trên là chỉ có lấy thông tin của 1 trang, vậy nên ta cần phải lặp lại các thao tác trên cho hết tất cả các trang có chứa thông tin cần lấy, ở đây nhóm đã lọc ra được có tất cả 12383 trang có chứa đầy đủ các thông tin mà chúng ta cần thu thập, vậy nên cần phải lặp lại 12383 lần để thu thập đầy đủ thông tin.

```
1  for page_number in range(1, 12384):
2  URL = 'https://hoidap.thuvienphapluat.vn/tim-theo-tu-van.html?page=' +
      str(page_number)
3  try:
4      get_page = requests.get(URL, timeout=30)
5      soup=BeautifulSoup(get_page.text, 'lxml')
```

3.3 Kết quả

Sau khi thực hiện xong việc Scraping data, kết quả thu được là **232321** cặp câu hỏi và câu trả lời, có ngày tháng được hỏi ở hai định dạng dữ liệu là CSV và JSON.

Dữ liệu ban đầu thu thập về là ở định dạng file CSV nhưng vì lý do cần phải xử lý định dạng lại cấu trúc hiển thị dữ liệu trên website nên nhóm đã chuyển đổi định dạng của file CSV sang file JSON.

Qua đó, nhóm đã giải quyết xong được bài toán đầu tiên cần phải giải quyết trong các bài toán nhỏ cần phải giải để có hoàn thành được hệ thống.

Chương 4

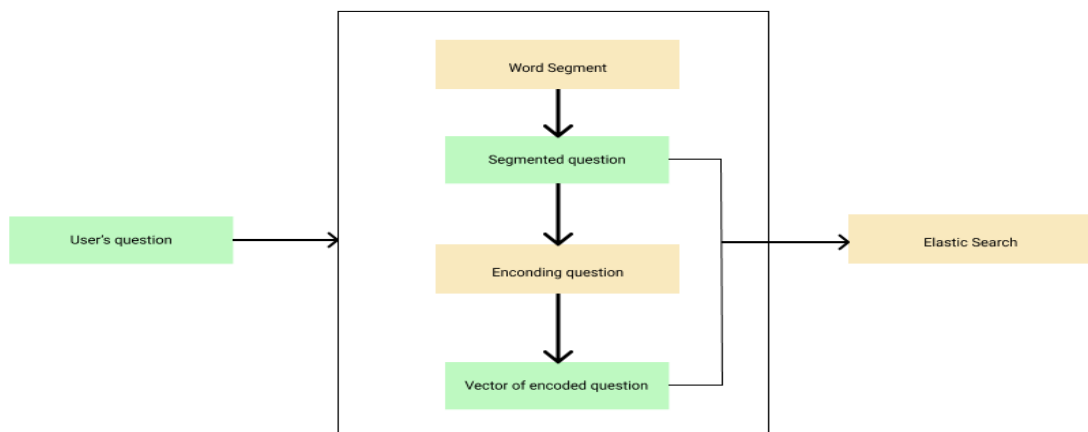
Kiến trúc phần Back-end

Ở chương này, nhóm sẽ tập trung giải thích về cách thức hoạt động của phần Back-end của hệ thống. Gồm các giai đoạn như sau:

- Xử lý dữ liệu thu thập: Mã hoá dữ liệu đã thu thập ở chương 3, lưu trữ dữ liệu vào trong cơ sở dữ liệu.
- Xử lý dữ liệu câu hỏi mà người dùng nhập vào trong hệ thống, cách mà hệ thống xử lý câu hỏi của người dùng và đưa ra kết quả cho người dùng.

4.1 Xử lý dữ liệu thu thập

Trong mục này, xử lý dữ liệu thu thập là tiến hành việc công việc phân loại từ câu hỏi, mã hoá câu hỏi từ dữ liệu bộ câu hỏi và câu trả lời đã thu thập được ở chương 3 và việc lưu trữ dữ liệu xuống Elastic Search.



Hình 4.1: Quy trình lưu dữ liệu xuống Elastic Search

4.1.1 Phân loại từ trong câu hỏi

Đầu tiên cần phải đề cập lại ở đây là có 3 cột dữ liệu đang có trong bộ dữ liệu đã được thu thập(dataset). Đó là cột câu hỏi, câu trả lời, ngày hỏi. Vì mục đích của hệ thống là hỏi đáp, đưa ra câu trả lời đúng và có mức độ liên quan gần nhất so với câu hỏi của người dùng nên đơn giản là chỉ ra được câu hỏi của người dùng nhập vào có bao nhiêu phần trăm là đồng nghĩa với câu hỏi có trong cơ sở dữ liệu. Vậy nên việc mã hoá dữ liệu cần phải làm là ở trên cột câu hỏi của dataset và câu hỏi của người dùng nhập vào từ giao diện website.

Trước khi mã hoá từ thì cần phải thực hiện một bước trước tiên đó là tách từ. Lý do cần phải thực hiện bước này trước tiên là vì tiếng việt là một ngôn ngữ không đơn giản, tiếng việt có tồn tại bên trong là các **từ ghép**. Vậy nên, nếu không thực hiện bước này trước tiên thì sẽ tạo ra một sự sai lệch về ngữ nghĩa trong khi mã hoá câu.

Ở đây thì nhóm sẽ thực hiện tách từ trong cột dữ liệu "Question" bằng PyVI như sau:

```
1 titles = [tokenize(doc["question"]) for doc in docs]
```

Dòng code này sẽ thực hiện công việc tách các từ ra, phân loại các từ đơn và từ ghép với nhau bằng dấu "_".

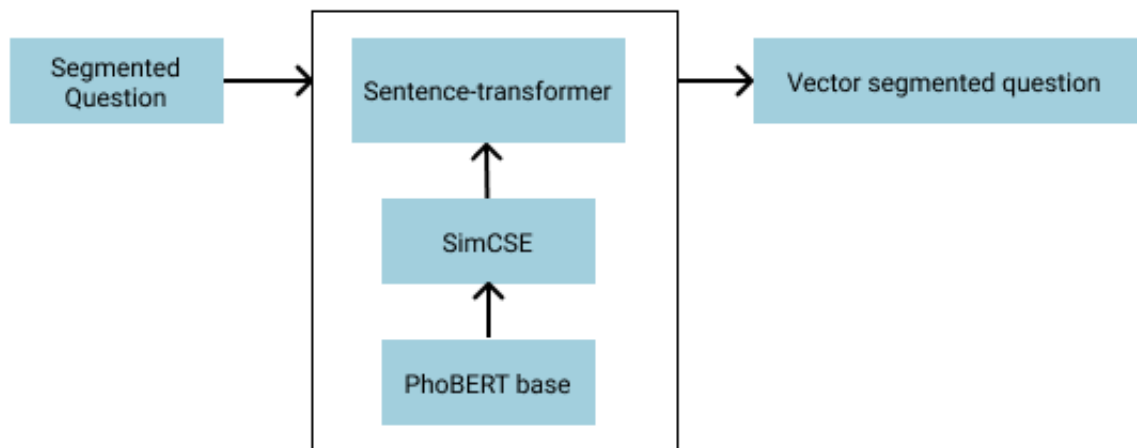
```
>>> from pyvi import ViTokenizer, ViPosTagger
>>> ViTokenizer.tokenize(u"Vuốt đèn đỏ xử lý thế nào?")
'Vuốt đèn_đỏ xử_ly thế_nào ?'
>>> |
```

Hình 4.2: Tokenize một câu hỏi từ dataset

Khi thực hiện tách từ xong, biến dữ liệu **titles** kiểu **str** sẽ lưu giữ tất cả các câu đã được tách từ. Theo như sơ đồ quy trình thì đây là thực hiện xong được bước Word Segment và được kết quả titles là Segmented question.

4.1.2 Mã hoá dữ liệu từ dataset

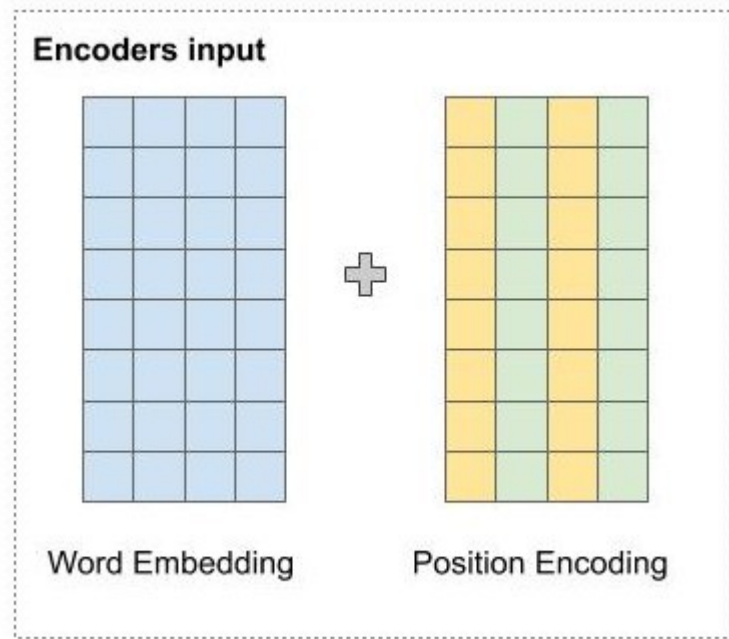
Kết quả sau giai đoạn phân tách sẽ được hệ thống mã hóa, chuyển hóa thành các vector biểu diễn thông tin. Tại đây, nhóm áp dụng mô hình SimCSE cho tiếng Việt[4] được phát triển dựa trên mô hình đào tạo sẵn là PhoBERT, với mục đích mã hóa, chuyển hóa thành các vector có thể biểu diễn đầy đủ ngữ nghĩa của từ của câu hỏi người dùng.



Hình 4.3: Quy trình thực hiện Encoding question

Tại đây, dữ liệu đầu vào sẽ được chức năng Mã hóa thực hiện như sau:

- Dữ liệu đầu vào sẽ được tính toán mã hóa dựa trên các từ, vị trí câu, vị trí từ trong câu với độ dài câu tối đa trong Position Embedding – mã hóa vị trí từ, là 512 tokens.
- Token đầu tiên trong mỗi chuỗi đều có giá trị là [CLS], đại diện cho cả câu trong tác vụ phân loại. Token [SEP] dùng để phân biệt các câu với nhau.
- Position Embedding – mã hóa vị trí từ trong câu: kiến trúc của BERT dựa trên kiến trúc của Transformers cho nên việc mã hóa vị trí từ trong câu cũng khá giống nhau. Vì Transformers có khả năng xử lý các từ song song nhưng nếu chỉ xử lý với mô hình Word Embedding sẽ không thể nào mô hình biết được vị trí của các từ để có thể hiểu ngữ nghĩa của câu. Vì vậy, cơ chế mã hóa vị trí từ trong



Hình 4.4: Encoders Input

câu xuất hiện để giải quyết vấn đề này bằng cách đưa thông tin vị trí của từ vào trong vector đại diện đầu vào.

Đối với việc mã hóa vị trí từ trong câu, PhoBERT hay BERT đều sử dụng công thức sau để tính toán vị trí[15]:

$$(PE)_{(pos, 2i)} = \sin(pos/10000^{(2i/d_{model})})$$

$$(PE)_{(pos, 2i + 1)} = \cos(pos/10000^{(2i/d_{model})})$$

Trong đó, pos là vị trí của từ trong câu, PE là giá trị phần tử thứ i trong ma trận Word Embedding có độ dài d_{model} .

Với việc mã hóa thành vector, SimCSE hay PhoBERT hay BERT đều dựa trên Transformers, sử dụng một cơ chế chung, cơ chế Attention., cho phép các từ khi được mã hóa sử dụng thông tin các từ liên quan đến nó.

Với mỗi từ, sẽ có ba vector tương ứng là Query, Key và Value. Các vector này được tính toán dựa trên ma trận Word Embedding.

Trong đó:

- Query: vector đại diện, chứa thông tin của từ được tìm kiếm.
- Key: vector chứa thông tin của các từ được so sánh với vector Query.

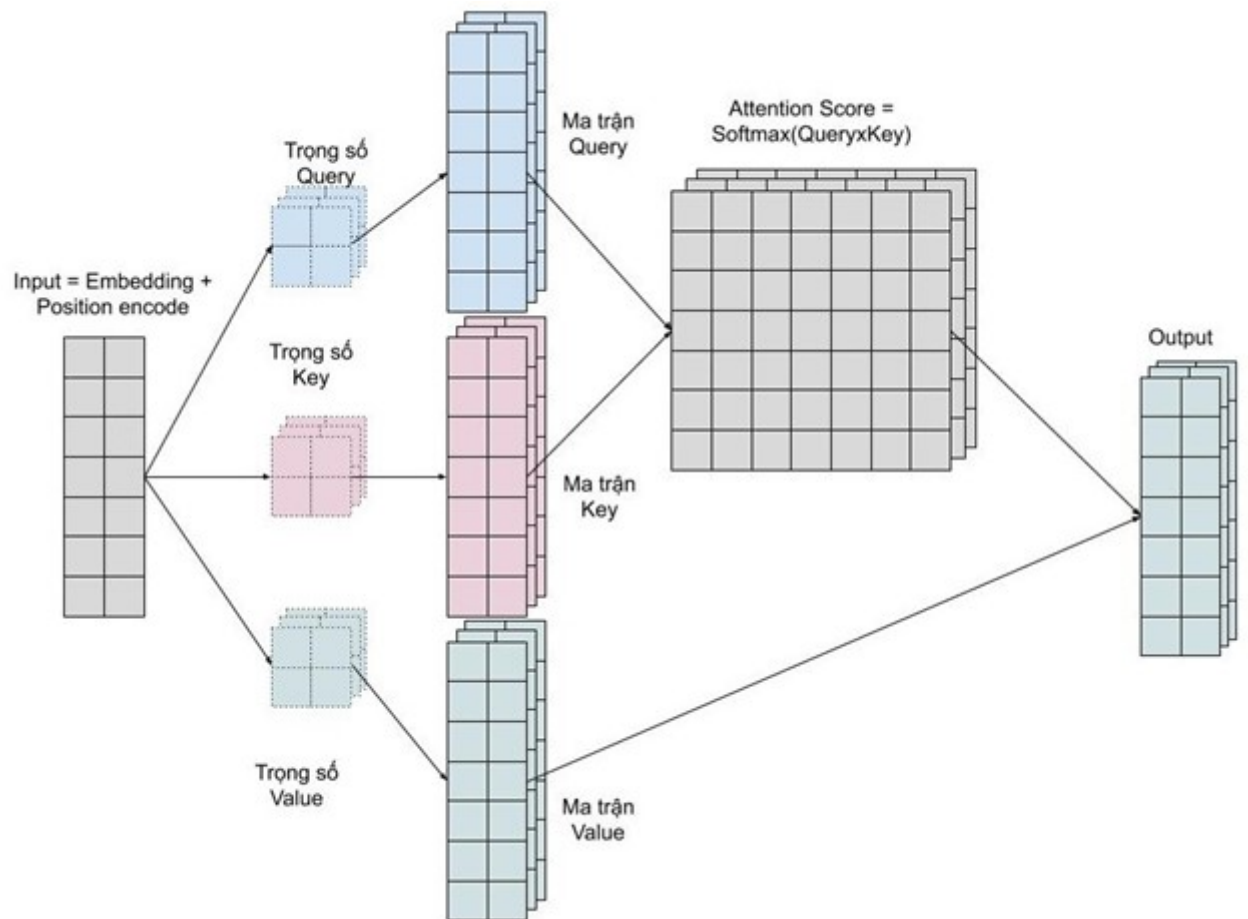
- Value: vector chứa thông tin về nội dung, ý nghĩa của các từ trong vector Key.

Các bước tính toán attention vector có thể tóm lại bằng ba bước:

- Bước 1: Tính ma trận Query, Key và Value dựa trên ma trận dữ liệu đầu vào là Word Embedding cộng với ma trận Position Embedding, tạo ra ba ma trận trọng số tương ứng với ba vector và nhân với ma trận dữ liệu đầu vào.
- Bước 2: Attention Score hay còn gọi là điểm tương quan hay trọng số, được tính bằng cách nhân hai ma trận Query và Key để so sánh mối liên hệ giữa các vector, cho ra được sự tương quan giữa chúng. Sau đó, dùng hàm Softmax để chuẩn hóa kết quả về đoạn $[0-1]$ với ý nghĩa 1 là vector Query tương quan với vector Key, 0 là không có sự tương quan nào giữa chúng.
- Bước 3: Tính trọng số kết quả đầu ra bằng cách nhân ma trận kết quả Attention Score với ma trận Value để biểu diễn được thông tin của từ.

Trong thành phần Encoder của Transformers hay của BERT, thành phần Encoder bao gồm hai phần chính là Multi-Head Attention và Feed-forward Network. Về bản chất, Multi-Head Attention là bao gồm nhiều lớp Self-Attention với cơ chế Attention đã được trình bày ở trên. Với mỗi Self-Attention sẽ cho ra được một mẫu kết quả khác nhau, nên việc sử dụng nhiều lớp Self-Attention sẽ giúp cho mô hình BERT cho ra kết quả chuẩn, có độ chính xác cao. Nó cho phép mô hình học được sự tương quan của từ dựa trên:

- Từ kế trước của một từ
- Từ kế sau của một từ
- Những từ liên quan, tương đồng với một từ.



Hình 4.5: Ma trận mã hoá dữ liệu của BERT[12]

Lấy ví dụ người dùng nhập câu hỏi "Vượt đèn đỏ thì xử lý thế nào", sau khi phân tách thành các từ khóa "Vượt", "đèn_đỏ", "xử_ly", "thế_nào", đưa vào mã hóa chuyển thành vector sẽ được nhận được ma trận vector của câu sau mã hóa với độ dài 768 tokens:

```
1 [ 1.9085e-01, 9.2255e-02, -8.8180e-02, 2.4886e-01, -1.8775e-02,  
2 ...  
3 1.8846e-01, 3.8965e-02, 5.0931e-02, 6.2369e-02, -2.3854e-01]]
```

Cuối cùng, thực hiện xong quy trình mã hoá thì ta nhận được kết quả là Vector of encoded question như trên mô hình quy trình lưu dữ liệu xuống Elastic Search ở trên.

4.1.3 Lưu dữ liệu đã mã hoá vào Elastic Search

Sau khi có được 2 cột dữ liệu cần thiết là Segmented Question và Vector of encoded question, ta tiến hành thêm dữ liệu vào trong Elastic Search.

Đầu tiên, những dữ liệu chúng ta cần phải lưu lại trong Elastic Search cần có là:

Tên thuộc tính	Kiểu dữ liệu
ID	text
Title	text
Question	text
Answer	text
Title_vector	text

Bảng 4.1: Bảng kiểu dữ liệu thêm vào cơ sở dữ liệu

Để định danh các kiểu dữ liệu này trước khi lưu xuống cơ sở dữ liệu, ta cần một file để định danh cấu trúc của dữ liệu như sau:

```
1 {
2   "mappings": {
3     "dynamic": "true",
4     "_source": {
5       "enabled": "true"
6     },
7     "properties": {
8       "id": {
9         "type": "text"
10      },
11      "title": {
12        "type": "text"
13      },
14      "question": {
15        "type": "text"
16      },
17      "answer": {
18        "type": "text"
19      },
20      "title_vector": {
21        "type": "dense_vector",
22        "dims": 768
23      }
24    }
25  }
26 }
```

Sau khi đã có file định danh cấu trúc cho dữ liệu, ta sử dụng thư viện Elasticsearch để bulk dữ liệu từ các biến lưu trữ thông tin về tách từ câu hỏi, vector tách từ câu hỏi vào trong Elastic Search.

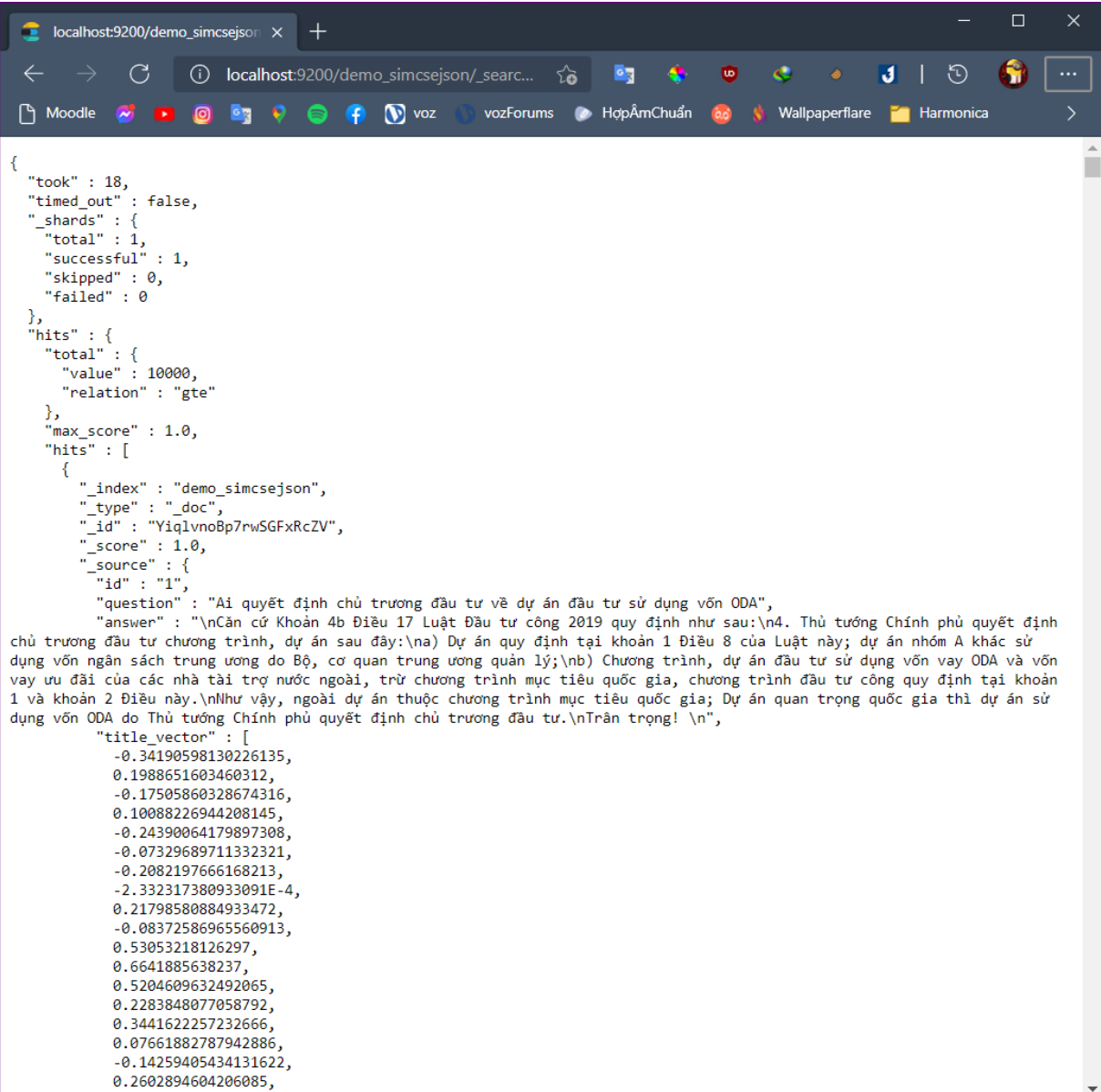
Lý do để phải thêm 2 cột dữ liệu mới là vì trong khi tính toán độ tương đồng về ngữ nghĩa của câu thì cần phải so sánh với nhau ở dạng vector với nhau, từ đó mà đưa ra được kết quả chính xác hơn.

4.1.4 Kết quả

Sau khi thêm dữ liệu thành công vào trong Elastic Search, để kiểm tra thông tin lưu trong Elastic Search thì ta thực hiện mở URL sau trên trình duyệt:

```
1 http://localhost:9200/demo_simcsejson/_search?pretty=true
```

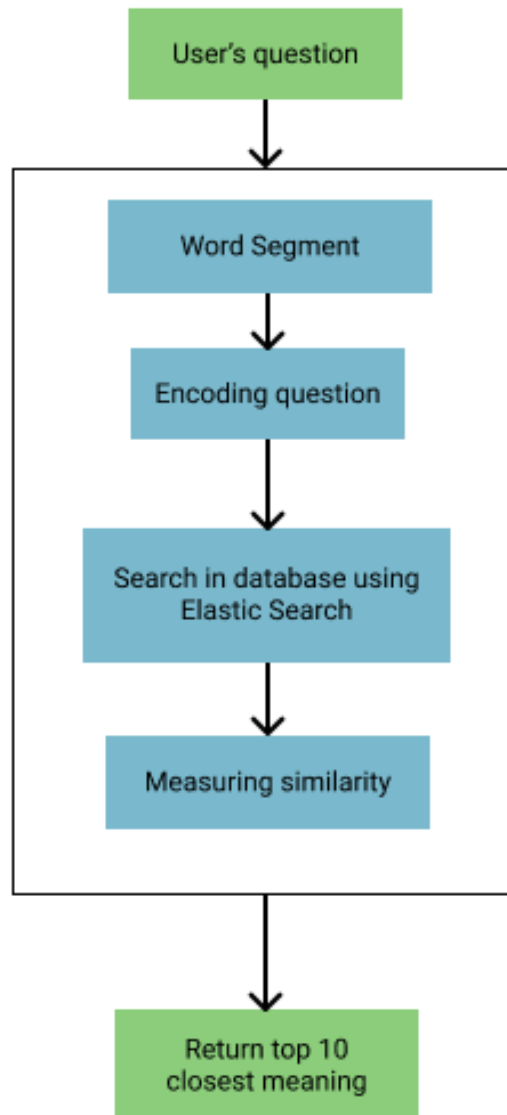
Sau khi truy cập vào URL trên, Elastic Search sẽ trả về dữ liệu được lưu trữ trong có dạng như sau:



```
{
  "took" : 18,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 10000,
      "relation" : "gte"
    },
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "demo_simcsejson",
        "_type" : "_doc",
        "_id" : "Yiq1vnoBp7rwSGFxRcZV",
        "_score" : 1.0,
        "_source" : {
          "id" : "1",
          "question" : "Ai quyết định chủ trương đầu tư về dự án đầu tư sử dụng vốn ODA",
          "answer" : "\nCăn cứ Khoản 4b Điều 17 Luật Đầu tư công 2019 quy định như sau:\n4. Thủ tướng Chính phủ quyết định chủ trương đầu tư chương trình, dự án sau đây:\n(a) Dự án quy định tại khoản 1 Điều 8 của Luật này; dự án nhóm A khác sử dụng vốn ngân sách trung ương do Bộ, cơ quan trung ương quản lý;\n(b) Chương trình, dự án đầu tư sử dụng vốn vay ODA và vốn vay ưu đãi của các nhà tài trợ nước ngoài, trừ chương trình mục tiêu quốc gia, chương trình đầu tư công quy định tại khoản 1 và khoản 2 Điều này.\nNhư vậy, ngoài dự án thuộc chương trình mục tiêu quốc gia; Dự án quan trọng quốc gia thì dự án sử dụng vốn ODA do Thủ tướng Chính phủ quyết định chủ trương đầu tư.\nTrân trọng! \n",
          "title_vector" : [
            -0.34190598130226135,
            0.1988651603460312,
            -0.17505860328674316,
            0.10088226944208145,
            -0.24390064179897308,
            -0.07329689711332321,
            -0.2082197666168213,
            -2.332317380933091E-4,
            0.21798580884933472,
            -0.08372586965560913,
            0.53053218126297,
            0.6641885638237,
            0.5204609632492065,
            0.2283848077058792,
            0.3441622257232666,
            0.07661882787942886,
            -0.14259405434131622,
            0.2602894604206085,
            0.4402265450545381
          ]
        }
      }
    ]
  }
}
```

4.2 Xử lý câu hỏi từ người dùng

Ở mục này, câu hỏi sẽ được người dùng nhập vào trong hệ thống và được hệ thống xử lý tìm kiếm bằng cách so sánh mức độ tương đồng về ngữ nghĩa thông qua việc mã hoá câu hỏi của người dùng nhập vào thành vector câu hỏi rồi so sánh với các vector câu hỏi có trong cơ sở dữ liệu.



Hình 4.6: Quy trình xử lý câu hỏi từ người dùng

Câu hỏi của người dùng sau khi nhập vào từ giao diện website sẽ được gửi từ web server đến phía Back-end của hệ thống. Khi nhận được dữ liệu câu hỏi, hệ thống sẽ bắt đầu thực hiện xử lý câu hỏi của người dùng theo từng bước là:

- Tách từ trong câu
- Mã hoá câu đã được tách từ
- Tìm kiếm trong cơ sở dữ liệu vector giống với vector câu hỏi nhất
- So sánh mức độ tương đồng với câu hỏi
- Trả về những câu hỏi có mức độ tương đồng ngữ nghĩa với câu hỏi của người dùng cao nhất

4.2.1 Tính toán mức độ tương đồng ngữ nghĩa giữa các câu

Ở mục này, nhóm sẽ giải quyết bài toán được đặt ra từ trước để có thể tiến hành hoàn thành hệ thống là bài toán tính mức độ tương đồng ngữ nghĩa giữa các câu.

Sau quá trình tìm hiểu các phương pháp được giới thiệu để tính toán được mức độ tương đồng giữa các vector với nhau, các phương pháp đều quy về một hướng đó là so sánh khoảng cách giữa các vector với nhau. Có 3 phương pháp so sánh vector với nhau mà được Google giới thiệu trong khoá học[0] là:

- Euclidean distance: Khoảng cách Euclidean giữa hai vector
- Cosine: Góc giữa hai vector
- Dot Product: Tích vô hướng giữa hai vector

Xét riêng tích vô hướng giữa hai vector và góc giữa hai vector, chúng sẽ là giống nhau khi độ dài hai vector cùng bằng nhau và bằng 1. Nhưng nếu độ dài của hai vector không bằng 1 thì việc tính toán bằng tích vô hướng giữa các vector sẽ trở nên phức tạp hơn so với khi sử dụng góc giữa các vector. Vậy nên nếu phải chọn phương pháp nào giữa tích vô hướng

và góc giữa các vector thì sử dụng góc giữa các vector sẽ là sự lựa chọn tốt hơn.

Bên cạnh đó, việc sử dụng khoảng cách Euclide trong không gian nhiều chiều sẽ được tính theo công thức là:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Nhìn vào công thức tính góc giữa các vector là:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Nếu xét trên không gian nhiều chiều, ta sẽ thấy được rằng công thức giữa Euclidean distance và Cosine Distance cũng sẽ đưa về một điểm chung đó là góc giữa hai vector. Vậy nên, sử dụng Cosine Distance sẽ là sự lựa chọn thích hợp trong trường hợp này. Ngoài ra, khi sử dụng chung với Elastic Search thì sẽ được Elastic Search hỗ trợ hàm được xây dựng sẵn trong việc tính toán.

Với A_i và B_i là thành phần của vector A và B tương ứng. Giá trị của cosine luôn nằm trong đoạn $[-1, 1]$:

- Giá trị cosine bằng 1, hai vector trùng nhau, suy ra, A và B giống nhau.
- Giá trị cosine càng tiến về -1, nghĩa là A và B khác nhau.

Như vậy, nếu giá trị cosine giữa hai vector càng tiến về 1, hai vector càng có sự tương đồng với nhau. Áp dụng vào hệ thống, ta suy ra được, nếu giá trị cosine giữa vector đại diện câu hỏi của người dùng và vector đại diện câu hỏi trong bộ dữ liệu càng gần 1, thì nội dung hai câu hỏi càng tương đồng.

Nhóm em đã tiến hành tìm kiếm sự tương đồng giữa vector đại diện câu hỏi người dùng và vector đại diện câu hỏi trong bộ dữ liệu bằng cách truyền vector đại diện câu hỏi người dùng vào Elastic Search, tính độ tương đồng Cosine Similarity nhờ vào hàm chức năng của Elastic Search. Cosine Similarity trên Elastic Search sau khi tính toán sẽ cộng thêm 1 để tránh

giá trị âm, như vậy giá trị cosine nằm trong $[1,2]$. Về mặt ý nghĩa không khác gì giá trị cosine nằm trong $[-1,1]$. Chúng em thu được một số kết quả như sau:

4.2.2 Trả về kết quả sau khi so sánh

Vì Elastic Search có hỗ trợ trong việc tính toán Cosine Distance, vậy nên trong khi tìm kiếm dữ liệu bằng Elastic Search, ngoài điều việc tìm kiếm các tách từ giống nhau thì ta chèn thêm một điều kiện nữa là có Cosine Distance nhỏ nhất.

Sau đây là đoạn script dùng để search dữ liệu trong Elastic Search:

```
1  "script_score": {
2    "query": {
3      "match_all": {}
4    },
5    "script": {
6      "source": "cosineSimilarity(params.query_vector, 'title_vector') +
7      1.0",
8      "params": {"query_vector": query_vector}
9    }
10 }
```

Khi nhìn vào đoạn script trên, `query_vector` là câu hỏi của người dùng nhập vào được tách từ (Word Segment) và `cosineSimilarity` là điều kiện search khi ánh xạ qua so với Query search của cơ sở dữ liệu quan hệ.

Câu hỏi của người dùng: “Vượt đèn đỏ thì xử lý thế nào”.

- Vector đại diện cho “Mức phạt hành vi vượt đèn đỏ” cho giá trị độ tương đồng bằng 1.8454137.
- Vector đại diện cho “Phạt bao nhiêu nếu ô tô vượt đèn đỏ” cho giá trị độ tương đồng bằng 1.7503384
- Vector đại diện cho “Hành vi điều khiển xe máy mà vượt đèn đỏ sẽ bị xử phạt bao nhiêu” cho giá trị độ tương đồng bằng 1.7825083.
- Vector đại diện cho “Sử dụng biển quảng cáo che mất đèn đỏ giao thông xử phạt thế nào” cho giá trị độ tương đồng 1.7325894.

Sau khi đã lấy ra được tập câu hỏi và trả lời thoả mãn được yêu cầu của hệ thống thì cần phải trả về kết quả ra để phía web server có thể lấy

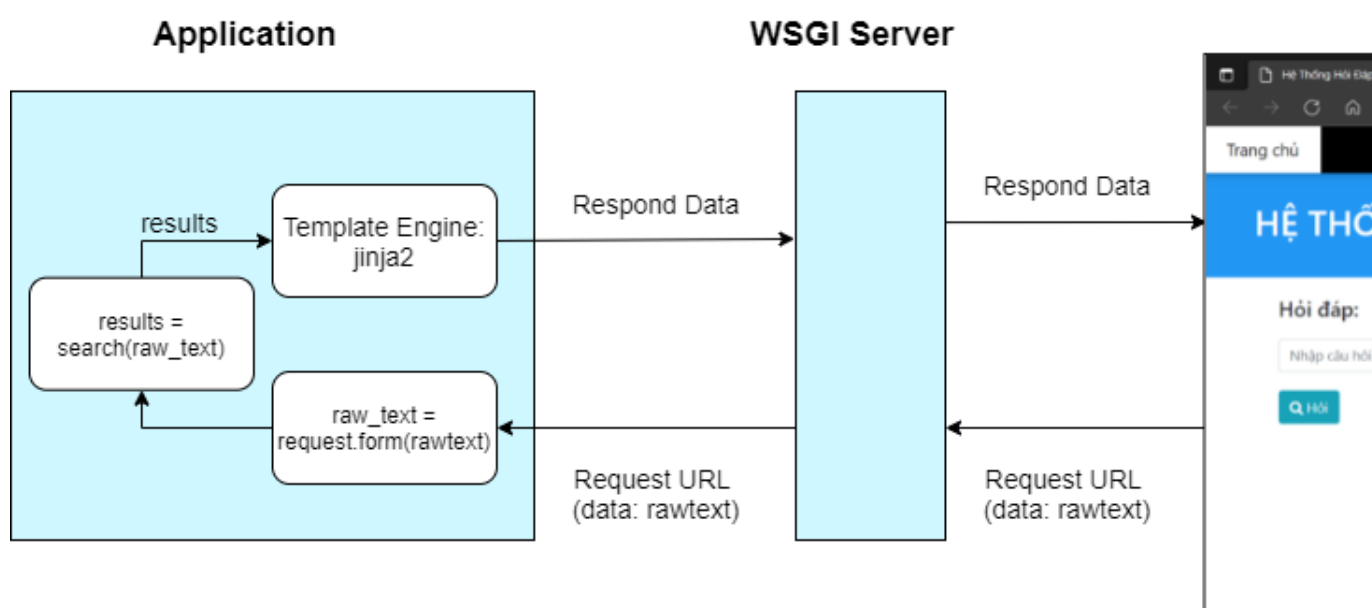
được kết quả trả về để gửi lại phía bên phía Front-end, hiển thị kết quả cho người dùng.

Chương 5

Kiến trúc phần Front-end

Ở chương này, nội dung trình bày sẽ là về kiến trúc xử lý thông tin trả về từ phía Back-end hệ thống đã xử lý xong các thao tác và trả ra kết quả các câu trả lời chính xác hoặc là có mức độ tương đồng nhiều nhất so với câu hỏi của người dùng đã nhập vào trong hệ thống.

Sau đây là sơ đồ quy trình của hệ thống khi đưa dữ liệu từ Back-end lên lại phía Front-end cho người dùng:



5.1 Tạo Web server bằng Flask

WSGI Server (Giao diện cổng máy chủ Web): ứng dụng Flask hoạt động trên máy chủ WSGI , và đây là cách thức mà máy chủ web giao tiếp với ứng dụng.

WSGI Server là máy chủ kích hoạt ứng dụng Web và việc truyền thông tin, với chức năng gọi lại cho ứng dụng. Mỗi khi nhận được Request

từ các client HTTP đến ứng dụng, ứng dụng đó sẽ được gọi bởi **WSGI Server**.

Việc xử lý yêu cầu (Request) sẽ diễn ra ở phía ứng dụng Web, sau đó ứng dụng sẽ gọi lại WSGI Server để trả lại kết quả được Request đến client.

Request dữ liệu: Khi Website request URL đến ứng dụng thông qua phương thức **POST**:

```
1 app.route('/', methods=['GET', 'POST'])
2 def index():
3     if request.method == 'POST':
4         raw_text = request.form['rawtext']
5         results = search(raw_text)
6         return render_template("results.html", results=results, raw_text=
raw_text)
7     else:
8         return render_template("index.html")
9
```

Ứng dụng sẽ lấy dữ liệu nhập được từ HTML template truyền vào biến `raw_text` thông qua hàm `request.form`:

```
raw_text = request.form['rawtext']
```

Ứng dụng sẽ gọi hàm `search` được import từ `utils.py` để thực hiện việc tìm kết quả để lưu vào biến `results`

Biến `results` và `raw_text` sẽ được truyền vào hàm `render_template` để gọi render cùng với HTML template

5.2 Parse dữ liệu từ dữ liệu trả về từ Back-end

Respond dữ liệu từ Back End lên Front End:

Template Engine: `jinja2`

Với việc sử dụng `jinja2` là một template engine, ta có thể hiển thị dễ dàng các biến `results` và `raw_text` đã được truyền vào trước đó bởi hàm `render_template` bằng cách gọi trực tiếp trong các HTML template:

```
1 <p class="input" style="...">Cau hoi da nhap: "{{raw\_text}}"</p>
2 <p class="input" style="...">{{ results[0][1] }}</p>
```

Kết quả nhận được là:

Câu hỏi đã nhập: "Có buộc phải lưu trữ số đăng ký cổ đông ở trụ sở công ty không?"
Có buộc phải lưu trữ số đăng ký cổ đông ở trụ sở công ty không?

Jinja2 còn có thể dùng để viết ra các hàm để xử lý dữ liệu được đổ vào Website:

```
1 {% if results != null %}
2     {{ answer(answer=results[0][2]) }}
3 {% endif %}
```

Kết quả nhận được là:

Điều 11 Luật Doanh nghiệp 2020 (Có hiệu lực từ 01/01/2021) quy định về các tài liệu được lưu trữ như sau:

- Tùy theo loại hình, doanh nghiệp phải lưu giữ các tài liệu sau đây:
- + Điều lệ công ty; quy chế quản lý nội bộ của công ty; số đăng ký thành viên hoặc số đăng ký cổ đông;
- + Văn bản bảo hộ quyền sở hữu công nghiệp; giấy chứng nhận đăng ký chất lượng sản phẩm, hàng hóa, dịch vụ; giấy phép và giấy chứng nhận khác;
- + Tài liệu, giấy tờ xác nhận quyền sở hữu tài sản của công ty;

....

- Doanh nghiệp phải lưu giữ các tài liệu quy định tại khoản 1 Điều này tại trụ sở chính hoặc địa điểm khác được quy định trong Điều lệ công ty; thời hạn lưu giữ thực hiện theo quy định của pháp luật.

Như vậy, số đăng ký cổ đông được lưu trữ tại trụ sở chính công ty hoặc địa điểm khác được quy định trong Điều lệ công ty.

Trân trọng!

```
1 {% for para in results if para[0] != 1 %}
2     {{group_QA(data_id = para[0] - 1,
3         question = para[1],
4         answer = para[2]) }}
5 {% endfor %}
```

Kết quả nhận được là:

Xem các câu hỏi và trả lời liên quan

Các câu hỏi liên quan:

1: Công ty cổ phần có bắt buộc có số đăng ký cổ đông không?

Xem câu trả Lỗi

2: Có bắt buộc công bố thông tin cuộc họp đại hội đồng cổ đông thường niên trên số giao dịch chứng khoán không?

Xem câu trả Lỗi

3: Số đăng ký cổ đông của công ty cổ phần phải có các nội dung nào?

Xem câu trả Lỗi

4: Biển hiệu công ty có bắt buộc phải được đặt tại trụ sở công ty không?

Xem câu trả Lỗi

5: Không lưu số đăng ký cổ đông công ty có bị phạt?

Xem câu trả Lỗi

6: Lưu giữ số đăng ký thành viên tại chi nhánh công ty được không?

Xem câu trả Lỗi

7: Ủy quyền cho cổ đông khác đi họp có phải lập văn bản không?

Xem câu trả Lỗi

8: Cổ đông sáng lập chuyển nhượng cổ phần có phải có cần phải thông báo thay đổi số đăng ký cổ đông?

Xem câu trả Lỗi

9: Cổ đông phổ thông thì có được sao chụp Điều lệ của công ty hay không?

Xem câu trả Lỗi

Chương 6

Kết luận

Tài liệu tham khảo

Tiếng Anh

- [4] DatCanCote. *SimCSE phát triển cho ngôn ngữ tiếng Việt*. URL: <https://github.com/DatCanCode/sentence-transformers>.
- [5] *Elastic Search*. 2021. URL: <https://viblo.asia/p/elasticsearch-la-gi-1Je5E8RmlnL>.
- [6] Gao, Tianyu, Yao, Xingcheng, and Chen, Danqi. “SimCSE: Simple Contrastive Learning of Sentence Embeddings”. In: *arXiv preprint arXiv:2104.08821* (2021).
- [0] Google. “Measuring Similarity from Embeddings”. In: (). URL: <https://developers.google.com/machine-learning/clustering/similarity/measuring-similarity>.
- [7] *Kỹ thuật Scraping webpages*. 2020. URL: <https://code.tutsplus.com/vi/tutorials/scraping-webpages-in-python-with-beautiful-soup-the-basics--cms-28211>.
- [8] Nguyen, Dat Quoc and Nguyen, Anh Tuan. “PhoBERT: Pre-trained language models for Vietnamese”. In: *Findings of EMNLP* (2020). URL: <https://arxiv.org/abs/2003.00744>.
- [9] Nils Reimers, Omar Sanseviero. “Sentence transformer”. In: (2020). URL: <https://huggingface.co/blog/sentence-transformers-in-the-hub>.
- [10] Online. *PhoBERT online*. 2021. URL: <https://viblo.asia/p/bert-roberta-phobert-bertweet-ung-dung-state-of-the-art-pre-trained-model-cho-bai-toan-phan-loai-van-ban-4P856PEWZY3>.
- [11] Online. *Thư viện Hỏi Đáp Pháp Luật*. URL: <https://hoidap.thuvienphapluat.vn>.

- [12] online. *Tìm hiểu mô hình Transformer*. 2021. URL: <https://pbcquoc.github.io/transformer/>.
- [13] Online. *Xây dựng Web API với Flask*. 2021. URL: <https://vimentor.com/vi/lesson/5-xay-dung-web-api-voi-flask>.
- [14] Tran, Viet Trung. “Python Vietnamese Toolkit”. In: (2021). URL: <https://github.com/trungtv/pyvi>.
- [15] Zhiheng Huang Davis Liang, Peng Xu and Xiang, Bing. “Improve Transformer Models with Better Relative Position Embeddings”. In: (). URL: <https://arxiv.org/abs/2009.13658>.